

DSA STUDY BLUEPRINT SERIES

122. Kruskals Algorithm Minimum Spanning Tree in Graph

Minimum Spanning Tree Construction using DSU (Kruskal's)

Section 10 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Kruskal's:** Sort all graph edges by weight.
- Iterate sorted edges; append to MST if they connect disjoint sets.

Memory & Execution Blueprint

Sorted edges: (1-2: weight 1), (2-3: weight 2), (1-3: cycle! skip) $\Rightarrow$ MST

C++ Implementation Reference

Standard code layout and syntax

```
int kruskalMST(int V, vector<Edge>& edges) {  
    sort(edges.begin(), edges.end(), [](auto& a, auto& b){ return a[2] < b[2]; });  
    vector parent(V), rank(V, 0);  
    for(int i=0; i<V; i++) parent[i] = i;  
    for(int i=0; i<edges.size(); i++)  
        if(!isSameParent(edges[i].u, edges[i].v))  
            unionEdges(edges[i].u, edges[i].v);  
    return parent;
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Sort Edges	$O(E \log E + E \alpha(V))$ time, $O(V)$ space

Key Practices & Gotchas

- **Important:** Sorting edges dominates the total runtime.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

Vertex popped	Adjacent edge checked	Current distance value	New path calculation	Relaxation action
U (Dist = 2)	U → V (Weight = 3)	V (Dist = 7)	New Dist = 2 + 3 = 5	Relax: Update V (Dist = 5)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.

